SOFTWARE REQUIREMENTS SPECIFICATION

PriceSmart

Smart Price Comparison and Product Tracking System

Programme	B.Tech Computer Engineering — 1st Year
Course	Fundamentals of Computer Systems and Networking
Prepared By	Group 9
College	ZCOER
Document Date	July 2026

Document Version 1.0

Table of Contents

1.	Introduction	3
2.	Overall Description	3
3.	Functional Requirements	4
4.	Non-Functional Requirements	5
5.	System Design Overview	5
6.	UI / UX Requirements	6
7.	Future Scope & Conclusion	7

Document Purpose

This Software Requirements Specification (SRS) describes the complete functional and non-functional requirements of the PriceSmart system. It is intended to be read by developers, the project guide, and viva examiners in order to understand the scope, design, and behaviour of the application.

1. Introduction

1.1 Purpose

This SRS describes the complete requirements for the PriceSmart system. It is intended for developers, the project guide, and viva examiners to understand what the system does, how it is structured, and the boundaries of its scope.

1.2 Scope

PriceSmart is a simplified web application that demonstrates price comparison and product tracking concepts without implementing production-grade features such as real-time data collection, browser extensions, cashback systems, or large-scale databases. The Minimum Viable Product (MVP) provides the following capabilities:

- Centralized price comparison from Amazon, Flipkart, and Croma using sample data
- Visual price history tracking using interactive graphs
- User account functionality with wishlist and a price-drop alert demo
- A web-based, responsive user interface

Out of Scope — the current version explicitly does **not** include real-time web scraping, a browser extension, or payment gateway integration. These are identified as future enhancements.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
MVP	Minimum Viable Product
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface

2. Overall Description

2.1 Product Perspective

PriceSmart is a standalone web application built on a conventional three-tier architecture. The **frontend** is developed using HTML, CSS, JavaScript and React for demonstration purposes. The **backend** is a Python Flask REST API, and the **database** layer uses SQLite. Price history and comparison visuals are rendered using Chart.js / Recharts.

User Browser → Frontend UI → Flask Backend API → SQLite Database

2.2 Product Functions

- User registration and login
- Product search by name or category (mobile, laptop, headphones, etc.)
- Display of detailed product information
- Price comparison table across three stores with a “Lowest Price” badge
- Price history graph (last 30 days) showing price increase / decrease trends
- Wishlist to save products of interest
- Price-drop alert (demo functionality)

2.3 User Classes and Characteristics

User Class	Description

End User	Can search, compare, add to wishlist and set alerts. No technical knowledge required.
Admin (Future)	Can add and manage products in the catalogue.

2.4 Operating Environment

- Operating System: Windows / Linux / macOS
- Browser: Latest version of Chrome or Firefox
- Runtime: Python 3.8 or higher, Flask framework
- Minimum Hardware: 2 GB RAM

3. Functional Requirements

The table below enumerates the functional requirements (FR) of PriceSmart, grouped by module. Each requirement is uniquely identified for traceability during development and testing.

Module	ID	Description
Authentication	**FR1.1**	Register with Name, Email, and Password
Authentication	**FR1.2**	Login with email/password; session stored in localStorage
Authentication	**FR1.3**	Logout functionality
Search	**FR2.1**	Search bar for product name
Search	**FR2.2**	Filter by category: All, Mobiles, Laptops, Headphones, Smartwatch
Search	**FR2.3**	Display product cards showing image, title, lowest price, store, discount, and rating
Comparison	**FR3.1**	Show prices for the same product from Amazon, Flipkart, and Croma
Comparison	**FR3.2**	Highlight the lowest price in green with a badge
Comparison	**FR3.3**	Example: iPhone 15 — Amazon 65 ,999, Flipkart 64 ,499 (Lowest), Croma 66,500
History	**FR4.1**	Store 30-day price history for each product
History	**FR4.2**	Display a line graph using Chart.js showing High / Low / Average
Wishlist	**FR5.1**	Add or remove a product from the wishlist; persist and show item count
Alert	**FR6.1**	Enter a target price; if current price ≤ target, show a notification

4. Non-Functional Requirements

ID	Category	Requirement
NFR1	Performance	Page load time under 3 seconds; search response under 1 second
NFR2	Usability	Clean UI that is responsive and intuitive to navigate
NFR3	Reliability	No application crash on invalid search input
NFR4	Security	Password hashing to be implemented in a real production system
NFR5	Portability	Runs on any system with a browser and Python installed
NFR6	Maintainability	Modular codebase that allows new stores to be added easily

5. System Design Overview

5.1 ER Diagram (Logical View)

Users(id, name, email, password) 1–N Wishlist(id, user_id, product_id, target_price) N–1 Products(id, name, category, image, rating, specs) 1–N Prices(id, product_id, store, price, original_price, date) + PriceHistory(id, product_id, price, date)

5.2 Database Schema (SQLite)

```
CREATE TABLE Users(id INTEGER PRIMARY KEY, name TEXT, email UNIQUE, password
TEXT); CREATE TABLE Products(id INTEGER PRIMARY KEY, name TEXT, category TEXT,
image_url TEXT, rating REAL, specs TEXT); CREATE TABLE Prices(id INTEGER PRIMARY
KEY, product_id INTEGER,   store TEXT CHECK(store IN
('Amazon','Flipkart','Croma')),   price INTEGER, original_price INTEGER, date
DATE); CREATE TABLE Wishlist(id INTEGER PRIMARY KEY, user_id INTEGER,   product_id
INTEGER, target_price INTEGER);
```

5.3 System Flow

1	User opens the Home page and views trending products
2	User enters a query, e.g. ‘iPhone’, and results are filtered
3	User clicks a product to open its Detail page
4	Backend fetches all store prices for the selected product_id
5	Frontend compares prices, computes the minimum, and highlights it
6	Backend returns the price-history array, which is rendered as a Chart.js graph
7	User may add the product to the wishlist or configure a price alert

6. UI / UX Requirements

Navigation Bar

- PriceSmart logo
- Central search bar
- Wishlist item count
- Login control

Home Page

- Hero section with the heading ‘Find Lowest Prices Across Stores’
- Category chips for quick filtering
- Responsive product grid

Product Page

- Back navigation button
- Product image on the left, details on the right
- Price comparison table
- Price history graph
- Price-alert configuration box

Colour Palette

Swatch	Name	Hex
	White (Background)	#ffffff
	Blue (Primary)	#2563eb

	Green (Lowest Price)	#16a34a

Responsive Behaviour

Product grid displays 4 columns on desktop and collapses to a single column on mobile devices.

7. Future Scope & Conclusion

7.1 Future Scope

- Real-time price scraping using BeautifulSoup
- Browser extension for on-page price comparison
- Email notifications for price-drop alerts
- AI-based product recommendations
- Coupon and cashback section
- Dark mode support

7.2 Conclusion

This SRS defines a realistic and achievable scope for a 10-day mini project that appears professional while remaining feasible within the given timeframe. It covers full-stack development, database design, and data visualization, giving the team a clear and structured path from requirements to implementation.

Prepared By: Group 9 | **College:** ZCOER | **Date:** July 2026